CS 411-W Lab 1: Version 1

Brennen Gabriel

Team Gold, Old Dominion University

October 10, 2025

Table of Contents

No table of figures entries found.

1 Introduction

Algebra frequently poses a challenge for students since conventional classroom instruction rarely ignites genuine interest; instead, it can foster frustration, disengagement, and a diminished sense of confidence in mathematics(Decker). To address this issue, any effective solution must be interactive, motivating, and adaptable to the individual learning pace—elements that transform abstract formulas into an engaging and meaningful exploration for students.

Rogue-Like Algebra addresses this challenge by integrating a classic turn-based, rogue-like adventure with algebraic problem-solving. Through strategic battles and incremental challenges, players apply equations and concepts to progress, making each mathematical victory feel akin to a gameplay triumph. The primary objective is to assist middle-school and high-school students in developing enduring confidence in their mathematical abilities by transforming algebra from a source of stress into an immersive, game-driven quest.

2 Rogue-Like Algebra

Rogue-Like Algebra is a proposed educational game to help individuals in improving their mathematics skills. This section outlines the purpose, design goals, and intended impact of the game.

2.1 Product Description

Rogue-Like Algebra is a turn-based educational game designed to transform algebra learning into an engaging and immersive experience. The project addresses common difficulties students encounter with algebraic concepts by integrating strategic dungeon-crawling mechanics with problem-solving tasks. Players apply algebra to perform actions such as attacking, defending, and progressing, while mistakes result in tangible consequences like reduced combat effectiveness or character penalties.

Procedurally generated dungeons and adaptive difficulty levels provide replayability and a personalized challenge for learners. Although primarily targeted at middle and high school students, the game remains accessible to all audiences. By offering a gamified alternative to rote memorization, Rogue-Like Algebra promotes confidence, creativity, and persistence in mathematics. The game is web-based and playable in any modern browser.

2.2 Key Product Features and Capabilities

Rogue-Like Algebra incorporates several features and capabilities designed to make learning algebra engaging and effective. The gameplay combines turn-based combat

with an immersive storyline and a variety of mini-games, ensuring that mathematical concepts are presented in an interactive format rather than through traditional memorization. Engagement is further enhanced through adaptive systems that scale the difficulty of problems based on player performance. Time limits and boss battles provide additional motivation, creating both urgency and a sense of accomplishment that encourages persistence.

The primary objective of the game is to gamify algebra and reduce the perceived complexity of learning it. By embedding mathematical problem-solving into core gameplay mechanics, Rogue-Like Algebra transforms abstract concepts into practical, repeatable actions. The game also addresses common learning challenges by rewarding players for solving increasingly difficult problems and completing achievements. In doing so, it provides both intrinsic and extrinsic motivation for continued practice. Also, procedural generation and multiple accepted approaches to problem-solving reduce the risk of rote memorization, ensuring that players focus on genuine understanding rather than mechanical repetition.

2.3 Major Components

Rogue-Like Algebra is built on several major components that provide both functionality and scalability for the project. Development takes place in Visual Studio Code, a powerful and extensible integrated development environment that is both free and widely supported. Version control is managed through Git and GitHub, which are considered industry standards and have been recommended by stakeholders to ensure reliable collaboration and code management. Continuous integration and deployment are handled through GitHub Actions and Workflows, allowing seamless automation that integrates directly with the repository.

The game is designed to run on both Windows and Linux operating systems. Windows was selected as the primary platform due to its popularity among end users, while Linux was chosen for its importance in powering mobile and embedded systems. Development relies on the Godot engine, which is free, open source, and supported by extensive tutorials and documentation. This engine is also highly extensible, making it well-suited for a project that requires flexibility. Rogue-Like Algebra uses Rust as its primary programming language because of its speed, safety, and modern features. GDScript is employed as a secondary language to handle edge cases where direct Rust integration is not feasible.

Several core systems are being developed to support gameplay. Character controllers will be created or adapted from available templates to provide smooth and responsive movement. A leaderboard system will be implemented to track player progress and encourage competition. Procedural generation will be used to create both dungeons and math problems, ensuring replayability and reducing reliance on rote memorization.

Finally, enemy artificial intelligence will be developed and tuned to provide dynamic and challenging encounters.

3 Identification of Case Study

Having established the key features and major components of Rogue-Like Algebra, this section identifies the case study by examining the intended audience and the purposes the game serves. Understanding who the game is designed for and what outcomes it seeks to achieve is essential for evaluating its educational and entertainment value.

3.1 Audience

The primary audience for Rogue-Like Algebra consists of middle and high school students between the ages of twelve and eighteen. These students are in the process of learning or reinforcing foundational algebraic principles and often benefit from interactive, game-based approaches that make abstract concepts more accessible. Within this group, the game also appeals to STEM-oriented players and puzzle enthusiasts who enjoy logical challenges and problem-solving.

The secondary audience includes parents and educators who may use the game as a supplemental resource to reinforce classroom learning. The game is also designed to attract casual players who enjoy indie games, as well as those with interests in logic and strategy. This broader group expands the reach of Rogue-Like Algebra while also highlighting its potential to engage learners outside of traditional educational settings.

3.2 Purposes Served

For the primary audience, the game is intended to assist in both learning and reinforcing fundamental algebraic principles. By embedding math challenges into engaging gameplay, Rogue-Like Algebra offers an effective means of delivering concepts in a way that captures attention and sustains interest. At the same time, the game provides a fun and rewarding experience that encourages persistence in the learning process.

For the secondary audience, the game serves to refresh and reinforce algebraic concepts for individuals who may not be actively studying mathematics. It also provides a repeatable and enjoyable game loop that maintains entertainment value regardless of educational goals. By balancing these two purposes, Rogue-Like Algebra succeeds in offering both a meaningful educational tool and a compelling gameplay experience for a wide range of users.

4 Glossary

4.1 Terms

1. **Adaptive Learning** – An educational method that adjusts content difficulty based on the learner's performance to personalize the learning experience.
2. **Algebraic Concepts** – Foundational mathematical principles involving variables, equations, expressions, and functions typically taught in middle and high school.
3. **Continuous Integration (CI)** – A development practice where developers frequently merge code changes into a shared repository, triggering automated builds and tests to detect issues early.
4. **Continuous Deployment (CD)** – A software release process that automatically delivers validated code changes to production, minimizing manual intervention and ensuring faster updates.
5. **GDScript** – A Python-like scripting language used in the Godot game engine, designed for creating game logic.
6. **Git** – A distributed version control system that tracks changes in source code during software development, enabling collaboration and history tracking.
7. **GitHub** – A cloud-based platform for hosting Git repositories. It offers tools for collaboration, issue tracking, code review, and CI/CD integration.
8. **Godot** – A free and open-source game engine used for developing 2D and 3D games, known for its flexibility and ease of use.
9. **IDE (Integrated Development Environment)** – A software suite that provides tools like a code editor, debugger, and compiler to streamline software development (e.g., VS Code).
10. **Procedural Generation** – A method of creating data algorithmically rather than manually, often used for generating levels, puzzles, or content dynamically in games.
11. **Rogue-like** – A subgenre of games typically featuring procedural generation, turn-based movement, and permanent death, emphasizing strategy and replayability.
12. **Rust** – A systems programming language known for performance, safety, and concurrency.
13. **Turn-Based Combat** – A gameplay mechanic where players and opponents take alternating turns to perform actions such as attacking or defending.
14. **VS Code (Visual Studio Code)** – A popular, lightweight code editor developed by Microsoft, used for writing and debugging code.

5 References

1. “About Us - DragonBox / WeWantToKnow.” Dragonbox.com, dragonbox.com/about.
2. Prodigy. “Prodigy Math | Prodigy Education.” Www.prodigygame.com, www.prodigygame.com/main-en/prodigy-math/.
3. “IXL - Membership.” Www.ixl.com, www.ixl.com/membership.
4. Steam. “Steam Database.” SteamDB, steamdb.info/.
5. “Top Roguelike Games by Sales and Revenue on Steam as of 2022 – Steam Marketing Tool.” Games-Stats.com, games-stats.com/steam/?tag=roguelike.
6. Decker-Woodrow, Lauren E, et al. The Impacts of Three Educational Technologies on Algebraic Understanding in the Context of COVID-19. Vol. 9, 1 Jan. 2023, p. 233285842311659-233285842311659, https://doi.org/10.1177/23328584231165919.
7. Aine, Sandip, and Maxim Likhachev. “Anytime Truncated D* : Anytime Replanning with Truncation.” Proceedings of the International Symposium on Combinatorial Search, vol. 4, no. 1, 20 Aug. 2021, pp. 2–10, https://doi.org/10.1609/socs.v4i1.18295
8. Guy, Stephen, and Ioannis Karamouzas. Guide to Anticipatory Collision Avoidance.
9. “NavigationMesh.” Godot Engine Documentation, 2025, docs.godotengine.org/en/stable/classes/class_navigationmesh.html
10. “Creating Your First 2D Game with A* Algorithm.” HackerEarth Blog, 6 Oct. 2016, hackerearth.com/blog/developers/creating-first-2d-game-algorithm/